```
    x = next(iter(loader))[0].to(device)

    # Generate noise for diffusion
    noise = torch.randn_like(x).to(device)

    # Generate output from diffusion model
    y = model(x, noise)

    # Calculate loss
    loss = loss_fn(y, x)

    # Backpropagate and update parameters
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Save output image every 100 steps
    if (step + 1) % 100 == 0:
        save_image(y.cpu(), f'output_{step + 1}.png')
```

This script sets up a diffusion model, loads a dataset, and trains the model on the dataset using mean squared error as the loss function. The script also saves an output image every 100 steps. Note that this is just a sample script and will need to be adapted to your specific use case.